```
clc;
clear;
close all;
```

1. SIGNAL PREPARATION AND NOISE CORRUPTION

```
Fs = 16000;                 % Sampling rate (Hz)
duration = 1.5;             % Duration (seconds)
t = 0:1/Fs:duration-1/Fs;

f1 = 300;
f2 = 800;
f3 = 2200;

envelope = sin(pi*t/duration).^2;

x_clean = (sin(2*pi*f1*t) + ...
           0.6*sin(2*pi*f2*t) + ...
           0.3*sin(2*pi*f3*t)) .* envelope;

% Additive White Gaussian Noise
target_SNR = 10;
speech_pwr = mean(x_clean.^2);
noise_pwr_req = speech_pwr / (10^(target_SNR/10));

rng(42);                    % Set random seed
noise = sqrt(noise_pwr_req) * randn(size(t));
x_noisy = x_clean + noise;
```

2. SPECTRAL SUBTRACTION ALGORITHM

```
N_frame = 256;
hop = 128;
win = hamming(N_frame)';
N_fft = 512;

alpha = 2.0;
beta = 0.02;

% Noise Spectrum Estimation from Initial Frames
N_noise_frames = 5;
noise_spectrum_sum = zeros(1, N_fft);

for i = 1:N_noise_frames
    idx = (i-1)*hop + (1:N_frame);

    noise_spectrum_sum = noise_spectrum_sum + ...
        abs(fft(x_noisy(idx).*win, N_fft));
end
```

```
N_est = noise_spectrum_sum / N_noise_frames;

% Overlap-Add Processing
num_frames = floor((length(t)-N_frame)/hop) + 1;

x_enhanced = zeros(1, length(t));
win_gain = zeros(1, length(t));

for m = 1:num_frames
    idx = (m-1)*hop + (1:N_frame);

    frame = x_noisy(idx).*win;
    Y = fft(frame, N_fft);

    Y_mag = abs(Y);
    Y_phase = angle(Y);

    % Spectral Magnitude Subtraction with Floor
    S_hat_mag = max(Y_mag-alpha*N_est, beta*Y_mag);
    S_hat = S_hat_mag.*exp(1i*Y_phase);

    frame_enhanced = real(ifft(S_hat, N_fft));

    x_enhanced(idx) = x_enhanced(idx) + ...
        frame_enhanced(1:N_frame).*win;

    win_gain(idx) = win_gain(idx) + win.^2;
end

valid_idx = win_gain > 1e-3;
x_enhanced(valid_idx) = ...
    x_enhanced(valid_idx)./win_gain(valid_idx);

% Calculate Input and Output SNR
SNR_in = 10*log10( ...
    sum(x_clean.^2)/sum((x_clean-x_noisy).^2));

SNR_out = 10*log10( ...
    sum(x_clean.^2)/sum((x_clean-x_enhanced).^2));
```

3. COMMAND WINDOW DISPLAY

```
fprintf('============================================\n');
fprintf(' EXPERIMENT 19: SPEECH ENHANCEMENT\n');
fprintf('============================================\n');

fprintf('Sampling Frequency: %d Hz\n', Fs);
fprintf('Signal Duration: %.2f seconds\n', duration);
fprintf('Target SNR: %.2f dB\n', target_SNR);
fprintf('Input SNR: %.2f dB\n', SNR_in);
fprintf('Output SNR: %.2f dB\n', SNR_out);
fprintf('SNR Improvement: %.2f dB\n', SNR_out-SNR_in);
```

```
fprintf('==========================================\n');
```

```
==========================================
 EXPERIMENT 19: SPEECH ENHANCEMENT
==========================================
Sampling Frequency: 16000 Hz
Signal Duration: 1.50 seconds
Target SNR: 10.00 dB
Input SNR: 9.99 dB
Output SNR: 15.22 dB
SNR Improvement: 5.23 dB
==========================================
```

4. SUBFIGURE PLOTTING AND EXPORT

```
figW = 8.5;
figH = 5.5;

% Subfigure 19A: Clean Speech Waveform
figA = figure('Units', 'centimeters', ...
    'Position', [2, 2, figW, figH]);

plot(t, x_clean, 'b', 'LineWidth', 1.2);
grid on;
box on;
title('Clean Speech Waveform');
xlabel('Time (s)');
ylabel('Amplitude');

exportgraphics(figA, 'Exp19_SubfigA_CleanSpeech.png', ...
    'Resolution', 600);

% Subfigure 19B: Noisy Speech Waveform
figB = figure('Units', 'centimeters', ...
    'Position', [11, 2, figW, figH]);

plot(t, x_noisy, 'r', 'LineWidth', 1.2);
grid on;
box on;
title(sprintf('Noisy Speech Waveform (SNR = %.2f dB)', SNR_in));
xlabel('Time (s)');
ylabel('Amplitude');

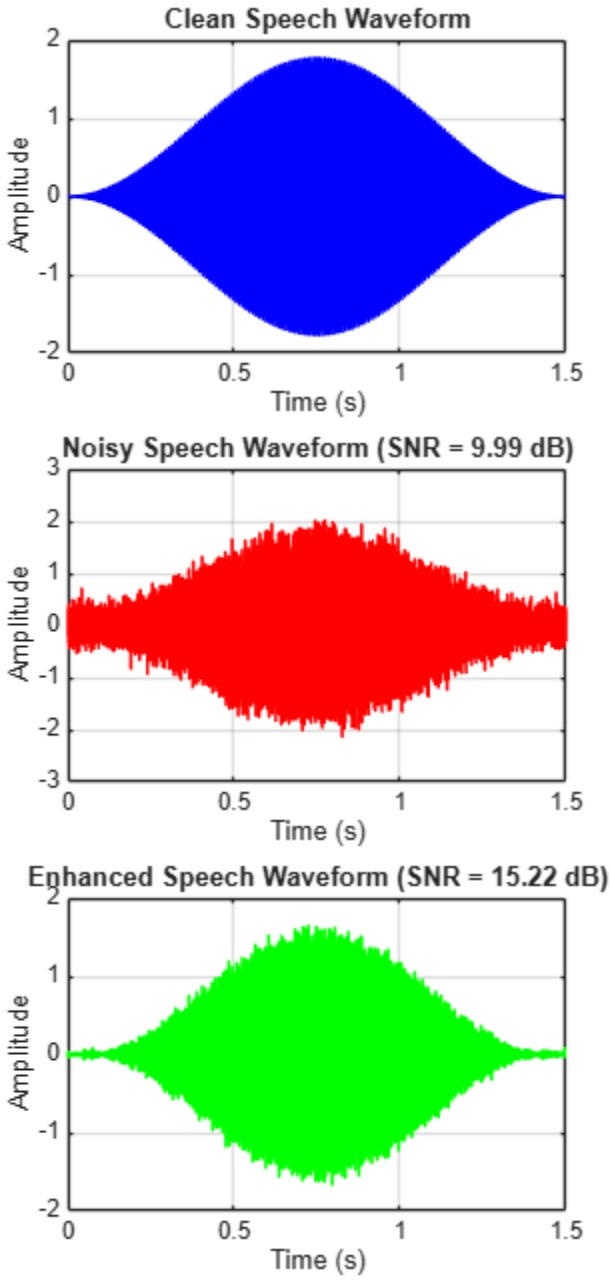
exportgraphics(figB, 'Exp19_SubfigB_NoisySpeech.png', ...
    'Resolution', 600);

% Subfigure 19C: Enhanced Speech Waveform
figC = figure('Units', 'centimeters', ...
    'Position', [20, 2, figW, figH]);

plot(t, x_enhanced, 'g', 'LineWidth', 1.2);
grid on;
```

```
box on;
title(sprintf('Enhanced Speech Waveform (SNR = %.2f dB)', SNR_out));
xlabel('Time (s)');
ylabel('Amplitude');

exportgraphics(figC, 'Exp19_SubfigC_EnhancedSpeech.png', ...
    'Resolution', 600);
```



Published with MATLAB® R2026b